OBSERVATION TASK

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: Task 3
Student Name: S.Deekshitha
Roll No.: A24126552117
Date: 30-08-2026

1. TITLE

Develop an Interactive To-Do List Application Using JavaScript, DOM Manipulation and Event Listeners

2. AIM

To develop an interactive To-Do List web application using JavaScript, DOM manipulation and event listeners that allows users to add, complete and delete tasks dynamically.

3. OBJECTIVE

The objectives of this experiment are:

- To provide a text box for entering a task and a button to add it.
- To dynamically create a new task item and add it to a list when the Add button is clicked.
- To provide a Complete button on each task that visually marks it as completed when clicked.
- To provide a Delete button on each task that removes it from the webpage when clicked.
- To display an appropriate message when the task list is empty.
- To demonstrate DOM element selection, dynamic element creation, and event handling.

4. THEORY

DOM manipulation refers to using JavaScript to read, add, modify or remove elements from a web page after it has loaded. Elements already present in the HTML are selected using methods such as document.getElementById(), while entirely new elements can be created at runtime using document.createElement() and inserted into the page with appendChild().

Event listeners connect user interactions, such as clicks or key presses, to JavaScript functions. addEventListener("click", handler) runs the given function whenever the target element is clicked, allowing the page to respond immediately without reloading.

In a to-do list application, each task is represented as a list item (<li>) built from smaller elements: a for the task text and a group of buttons for actions. Clicking Add Task reads the input value, builds this structure dynamically, and appends it to the task list. Clicking Complete toggles a CSS class (such as .completed) on the task text to apply a strikethrough style, using classList.toggle(). Clicking Delete removes the task's <li> element from the DOM using remove().

An emptyMessage element is shown or hidden based on whether the task list currently has any children, giving the user clear feedback when there are no tasks.

5. ALGORITHM / PROCEDURE

- Create an HTML file with a text input for the task, an Add Task button, an empty-state message, and an empty <ul> to hold tasks.
- Link an external CSS file to style the container, input, buttons and task items.
- Link an external JavaScript file to the HTML page.
- Select the input field, Add button, task list and empty-message elements using getElementById().
- Define a function to show or hide the empty-message based on the number of tasks in the list.
- Define an addTask() function that reads and trims the input value, and alerts the user if it is blank.
- Inside addTask(), dynamically create a list item, a span for the task text, and Complete/Delete buttons using createElement().
- Attach a click event listener to the Complete button that toggles a 'completed' class on the task text.
- Attach a click event listener to the Delete button that removes the list item from the page.
- Append the buttons and text span to the list item, and append the list item to the task list.
- Clear the input field and update the empty-state message after adding a task.
- Attach the addTask() function to the Add button's click event and to the Enter key on the input field.
- Open the HTML file in a browser and test adding, completing and deleting multiple tasks.
- Verify that the empty-state message appears when all tasks are deleted.

6. PROGRAM

index.html

```
cal.html > html > body > div.container
<!DOCTYPE html>
<html>
<head>
    <title>My To-Do List</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="container">
        <h1>My To-Do List</h1>

        <input type="text" id="taskInput" placeholder="Enter a task">
        <button id="addTask">Add Task</button>

        <ul id="taskList"></ul>

        <p id="emptyMessage">No tasks available.</p>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

style1.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
    text-align: center;
    padding: 40px;
}

.container {
    width: 500px;
    margin: auto;
    background-color: white;
    padding: 25px;
    border-radius: 10px;
    box-shadow: 0 0 10px gray;
}

h1 {
    margin-bottom: 20px;
}

#taskInput {
    width: 300px;
    padding: 10px;
    font-size: 16px;
}

#addTask {
    padding: 10px 15px;
    cursor: pointer;
}

ul {
    list-style: none;
    padding: 0;
}
```

```css
li {
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin: 10px 0;
    padding: 10px;
    background-color: #eeeeee;
    border-radius: 5px;
}

.taskText {
    flex: 1;
    text-align: left;
}

.completed {
    text-decoration: line-through;
    color: gray;
}

button {
    margin-left: 5px;
    padding: 7px 10px;
    cursor: pointer;
}
```

script.js

```js
const taskInput = document.getElementById("taskInput");
const addTaskButton = document.getElementById("addTask");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

addTaskButton.addEventListener("click", function () {

    const task = taskInput.value.trim();

    if (task === "") {
        return;
    }

    const listItem = document.createElement("li");

    const taskText = document.createElement("span");
    taskText.textContent = task;
    taskText.className = "taskText";

    const completeButton = document.createElement("button");
    completeButton.textContent = "Complete";

    completeButton.addEventListener("click", function () {
        taskText.classList.toggle("completed");

        if (taskText.classList.contains("completed")) {
            completeButton.textContent = "Completed";
        } else {
            completeButton.textContent = "Complete";
        }
    });
```

```
32
33      const deleteButton = document.createElement("button");
34      deleteButton.textContent = "Delete";
35
36      deleteButton.addEventListener("click", function () {
37          listItem.remove();
38          updateEmptyMessage();
39      });
40
41      listItem.appendChild(taskText);
42      listItem.appendChild(completeButton);
43      listItem.appendChild(deleteButton);
44
45      taskList.appendChild(listItem);
46
47      taskInput.value = "";
48
49      updateEmptyMessage();
50  });
51
52  function updateEmptyMessage() {
53      if (taskList.children.length === 0) {
54          emptyMessage.style.display = "block";
55      } else {
56          emptyMessage.style.display = "none";
57      }
58  }
59
60  updateEmptyMessage();
```

7. CODE EXPLANATION

Code	Explanation
document.getElementById()	Selects existing DOM elements such as the input box, Add button, task list and empty message.
taskInput.value.trim()	Reads the text entered by the user and removes leading/trailing whitespace.
document.createElement()	Dynamically creates new elements (list item, span, buttons) not present in the original HTML.
classList.toggle("completed")	Adds the 'completed' class if absent or removes it if present, toggling the strikethrough style.
li.remove()	Removes the task's list item element from the DOM when Delete is clicked.
appendChild()	Inserts a created element into the page as a child of another element (building the task item, then adding it to the list).
addEventListener("click", fn)	Registers a function to run when a button (Add, Complete, Delete) is clicked.
addEventListener("keydown", fn)	Allows a task to be added by pressing the Enter key inside the input field.
updateEmptyState()	Shows or hides the 'No tasks available' message based on whether the task list has any items.

Code	Explanation
taskList.children.length	Counts the current number of task items to determine if the list is empty.

8. TEST CASES AND EXECUTION DATA

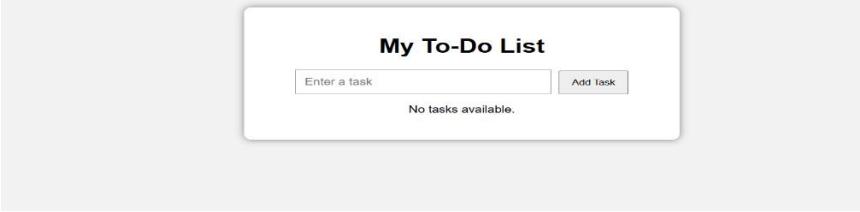
Each feature was tested by interacting with the application in the browser.

Test Case	Action	Expected Result	Status
TC-01	Add a task with text entered	New task appears in the list; input is cleared	Pass
TC-02	Click Add Task with empty input	Alert is shown; no task is added	Pass
TC-03	Press Enter after typing a task	Task is added, same as clicking Add Task	Pass
TC-04	Click Complete on a task	Task text shows a strikethrough style	Pass
TC-05	Click Complete again on the same task	Strikethrough style is removed (toggle)	Pass
TC-06	Click Delete on a task	Task is removed from the list	Pass
TC-07	Delete all tasks	"No tasks available." message is displayed	Pass

9. OUTPUT

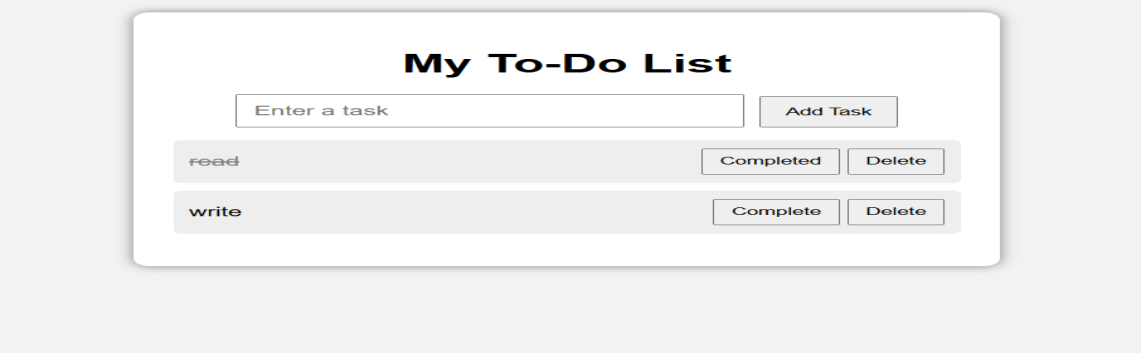
Output 1: Empty To-Do List

On first loading the page, the container shows the title, input box, Add Task button, and the "No tasks available." message since the list is empty.



Output 2: Tasks Added, Completed and Deleted

After adding a task, the item is displayed with its text and Complete/Delete buttons, ready to be marked complete or removed.



10. RESULT

Thus, an interactive To-Do List application was successfully developed using JavaScript, DOM manipulation and event listeners. Tasks could be added dynamically through the input box, marked as completed with a strikethrough style, and removed from the list, with the empty-state message displayed correctly whenever the list had no tasks. The application was verified using multiple test cases.